

TRIBHUVAN UNIVERSITY PULCHOWK CAMPUS



Project Report on “CV DATABASE”

Submitted To:

Department of Electronics and Computer Engineering

Submitted By:

Darpan Giri (080BCT024)

Kushal Gautam (080BCT040)

Lavraj Karn (080BCT043)

Mahesh Bhandari (080BCT045)

Abstract

This report describes the implementation of a Database Management System for CV Management (CV-MDS). It was developed for storing, managing, and updating the professional data that job applicants and recruitment departments generate. This report mainly includes the background and development process of the project. The main steps in the development process are identifying the entities (such as Users, Education, Experience, and Skills) and the relationships among them, and presenting them in graphical form with an ER diagram. It also includes the conversion of the conceptual model into a relational model, the application of normalization (up to 3NF) to ensure data integrity, and some of the SQL queries used in the implementation of the MySQL Database. This project is also useful for HR departments, recruitment agencies, and other similar organizations looking to transition from traditional document-based storage to a structured relational database.

Acknowledgement

We are very pleased to present the report on "CV Management Database System" which has been prepared for the partial fulfillment of the requirements for the degree of Bachelor in Computer Engineering (BCT).

We would like to express our gratitude and sincere thanks to our teacher for his/her guidance and support in the preparation of our report. We would also like to extend our gratitude to all of them who have helped us during the preparation of our report, i.e., our friends and family. Finally, we would like to acknowledge our college management for providing the materials and resources that were required for the preparation of this report.

We would take all the responsibility for our possible mistakes in our research report and we will try to resolve them.

Mahesh Bhandari

Darpan Giri

Kushal Gautam

Lav Raj Karna

TABLE OF CONTENTS

1. Introduction	7
1.1 Background	7
1.2 Problem Statement	7
2. Objective	8
3. Literature Review	9
3.1 Overview of Database Management Systems	9
3.2 Relational Model and Normalization	9
3.3 CV and Resume Management Systems	9
3.4 ER Modeling in Database Design	9
3.5 MySQL as a Platform	10
4. Methodology / System Design	11
4.1 Entity-Relationship Model	11
4.1.1 Entities and Entity Set	11
4.1.2 Attributes	11
4.1.3 Relationships	12
4.2 ER Diagram	12
4.3 ER Model to Relational Model Conversion	13
4.3.1 Step 1: User and Experience (One-to-Many)	13
4.3.2 Step 2: User and Education (One-to-Many)	14
4.3.3 Step 3: User and Skill (Many-to-Many)	14
4.3.4 Step 4: User and Certificate (One-to-Many)	15
4.3.5 Step 5: User and Project (One-to-Many)	15

4.4 Normalization up to 3NF	16
4.4.1 First Normal Form (1NF)	16
4.4.2 Second Normal Form (2NF)	16
4.4.3 Third Normal Form (3NF)	16
5. Implementation Details	18
5.1 Development Environment	18
5.1.1 Programming Languages, Tools, Libraries and Frameworks Used	18
5.1.2 Hardware and Software Specifications	18
5.2 Interface and Visualization	18
5.3 Deployment and Execution	19
5.3.1 System Setup	19
5.3.2 Execution Steps	19
5.3.3 Local / Server / Cloud Setup	20
5.4 Table Creation (DDL)	20
5.5 Data Insertion	23
6. Results and Discussion	27
6.1 Database Schema	27
6.2 Normalization Outcomes	27
6.3 Data Integrity and Referential Integrity	27
6.4 Query Performance and Usability	27
6.5 Challenges Encountered	27
7. Conclusions and Recommendations	29
7.1 Conclusions	29
7.2 Recommendations	29

References	31
Book References	31
Web References	31

1. Introduction

1.1 Background

In the current professional landscape, managing a large volume of resumes (CVs) manually is a daunting task for HR departments and recruitment agencies. Traditional methods — such as storing Word documents or PDFs in nested folders — result in significant data silos. Without a structured Database Management System (DBMS), searching for a specific candidate based on a niche skill set or a specific range of experience becomes a time-consuming manual process. This project proposes a CV Management Database System to digitize, structure, and streamline the recruitment workflow.

1.2 Problem Statement

The traditional approach to CV management presents several critical challenges that hinder efficient operations in recruitment:

- **Data Redundancy:** Storing multiple versions of the same resume leads to wasted storage and confusion.
- **Difficulty in Data Retrieval:** Finding candidates with specific qualifications (e.g., "5+ years in Java") requires opening every file manually.
- **Inefficient Utilization of Manpower:** Not maintaining a DBMS can lead to inefficient utilization of manpower, as staff may need to spend more time on manual data entry and management tasks, reducing their productivity and preventing them from focusing on other important tasks like interviewing and talent acquisition.

2. Objective

The main objective of this project is to provide proper solutions to the existing problems in candidate data management due to traditional approaches. Specific objectives include:

- **Centralized Database:** Maintain a centralized database for all candidate profiles and professional histories.
- **Data Security:** Provide proper security of data by restricting access to sensitive contact information.
- **Efficient Data Management:** Achieve efficient data management through the use of relational tables.
- **Regular Data Updates:** Allow candidates to add new skills or experiences easily through regular data updates.
- **Easy Data Retrieval:** Enable easy retrieval of data using optimized SQL queries for recruitment filtering.

3. Literature Review

3.1 Overview of Database Management Systems

A Database Management System (DBMS) is software that enables the creation, management, and querying of structured data. According to Ramakrishnan and Gehrke (2003), a DBMS provides a unified environment that supports data independence, query optimization, concurrency control, and recovery. Relational DBMS (RDBMS) systems, in particular, have become the cornerstone of modern information systems due to their adherence to mathematical set theory and the relational model proposed by Codd (1970).

3.2 Relational Model and Normalization

The relational model organizes data into tables (relations), where each row represents a tuple and each column an attribute. Silberschatz, Korth, and Sudarshan (2010) elaborate on normalization as the process of structuring a database schema to reduce redundancy and improve integrity. The Normal Forms — 1NF, 2NF, and 3NF — are progressively stricter rules that eliminate partial and transitive dependencies among attributes, ensuring each table contains only directly related data.

3.3 CV and Resume Management Systems

Several studies have explored digitizing human resource processes. Singh (2010) demonstrates that transitioning from file-based storage to relational databases for personnel management significantly improves query response times and reduces data inconsistencies. Modern HR systems like SAP SuccessFactors and Oracle HCM use complex relational schemas to store employee and applicant data. However, for academic and small-to-medium organizations, a lightweight MySQL-based system is a practical and cost-effective alternative.

3.4 ER Modeling in Database Design

Chen (1976) introduced the Entity-Relationship (ER) model as a way to conceptually design databases before physical implementation. The ER model identifies entities, attributes, and relationships, which are then mapped to relational tables through a systematic procedure. This model-first approach is particularly useful for complex domains such as recruitment, where entities like Users, Education, Experience, Skills, Projects, and Certifications interact in diverse ways.

3.5 MySQL as a Platform

MySQL is a widely adopted open-source RDBMS maintained by Oracle Corporation. Its performance, scalability, ease of use, and community support have made it a preferred platform for academic projects and production systems alike. In combination with XAMPP and phpMyAdmin, MySQL provides an accessible local development environment suitable for the implementation of a CV database system.

4. Methodology / System Design

4.1 Entity-Relationship Model

The E-R model is one of the most popular conceptual models used for designing a database. It views the real world as a set of basic objects, their characteristics, and associations among these objects. The entities, attributes, and relationships are the basic constructs of an E-R model.

4.1.1 Entities and Entity Set

An entity is an object or a concept that is defined by the enterprise as having an independent existence. An entity set is a set of entities of the same type that share the same properties or attributes. In this project, five core entities are identified:

- **User:** The job seeker / profile owner
- **Education:** Academic background records
- **Experience:** Professional work history
- **Skill:** Technical or soft skills
- **Project:** Portfolios or specific works
- **Certification:** Certificates of different works

4.1.2 Attributes

Attributes are the information that explains the properties of an entity. Attributes are the descriptive properties possessed by all members of an entity set. Each entity may have its own value for each attribute. Below is the complete attributes table for all entities (primary keys are underlined):

Entity	Attributes
User	<u>user_id</u> (PK), full_name, email, phone, address, linkedin_url, gender, date_of_birth
Education	<u>education_id</u> (PK), institution, degree, major, gpa, start_date, end_date
Experience	<u>experience_id</u> (PK), company_name, job_title, start_date, end_date, job_description, location
Skill	<u>skill_id</u> (PK), skill_name, skill_type
Project	<u>project_id</u> (PK), project_name, description, project_link, technologies_used
Certificate	<u>certification_id</u> (PK), certification_name, issue_date, expiry_date, issuing_organization

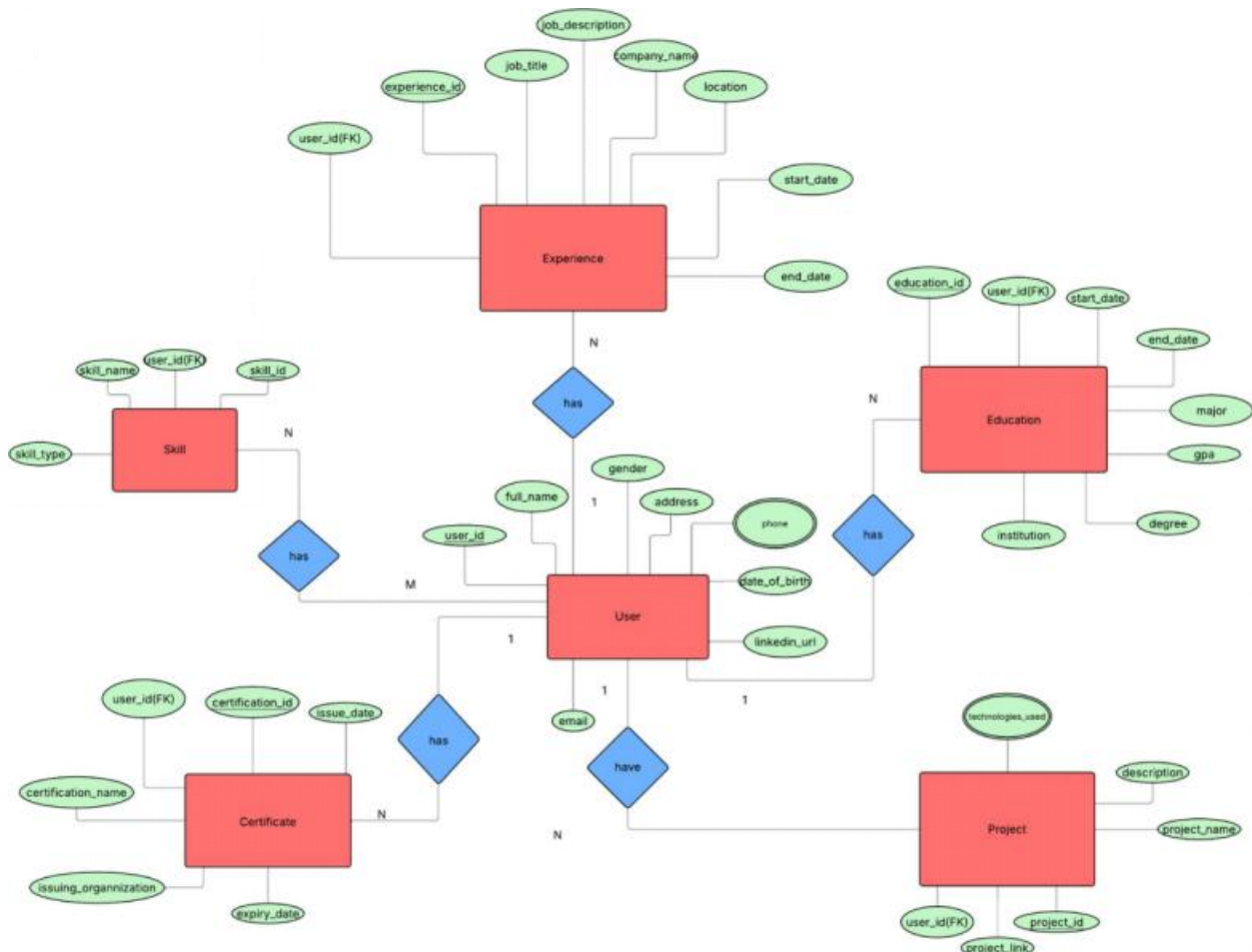
4.1.3 Relationships

Association between two or more entities is called a relationship. Relationships are represented by a diamond shape in the ER diagram. The following relationships exist in the CV Management System:

- **User — Education:** One-to-Many (a user can have multiple education records)
- **User — Experience:** One-to-Many (a user can have multiple experience entries)
- **User — Skill:** Many-to-Many (a user can have multiple skills; a skill can belong to multiple users)
- **User — Project:** One-to-Many (a user can have multiple projects)
- **User — Certificate:** One-to-Many (a user can have multiple certifications)

4.2 ER Diagram

The ER diagram below provides a complete graphical representation of the entities, their attributes, and the relationships between them in the CV Management Database System:

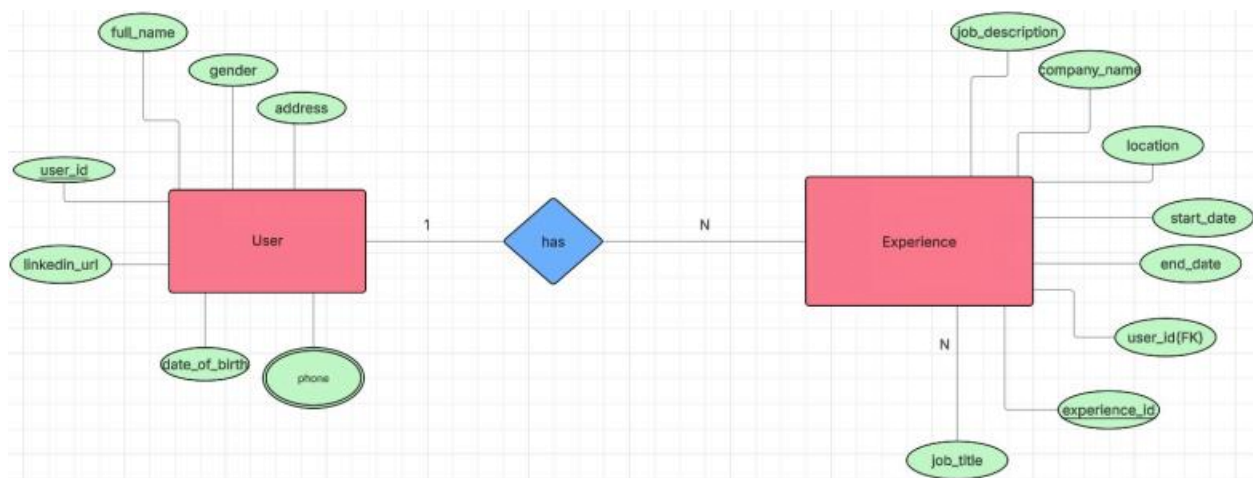


4.3 ER Model to Relational Model Conversion

The ER diagram of the CV database was successfully transformed into a relational model by converting each entity into a table and representing relationships using primary and foreign keys. Multivalued attributes were handled through separate relations, and the one-to-many relationships were maintained using foreign key constraints.

4.3.1 Step 1: User and Experience (One-to-Many)

There is a One-to-Many relationship between User and Experience. Therefore, `user_id` is used as a Foreign Key in the Experience table.



User Table:

user_id (PK)	gender	full_name	address	linkedin_url	date_of_birth

User_Phone Table (multivalued attribute):

Since a user can have multiple phone numbers, a separate User_Phone table is created to handle the multivalued attribute phone. It contains `user_id` as a foreign key referencing the User table.

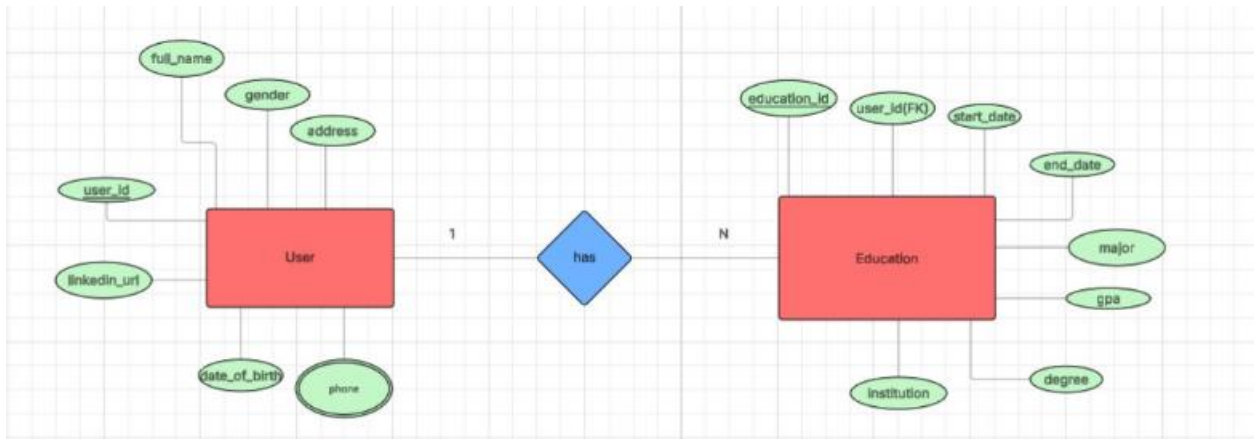
user_id (FK)	phone

Experience Table:

experience_id (PK)	job_title	job_description	company_name	location	start_date	end_date	user_id (FK)

4.3.2 Step 2: User and Education (One-to-Many)

There is a One-to-Many relationship between User and Education. Therefore, user_id is used as a Foreign Key in the Education table.

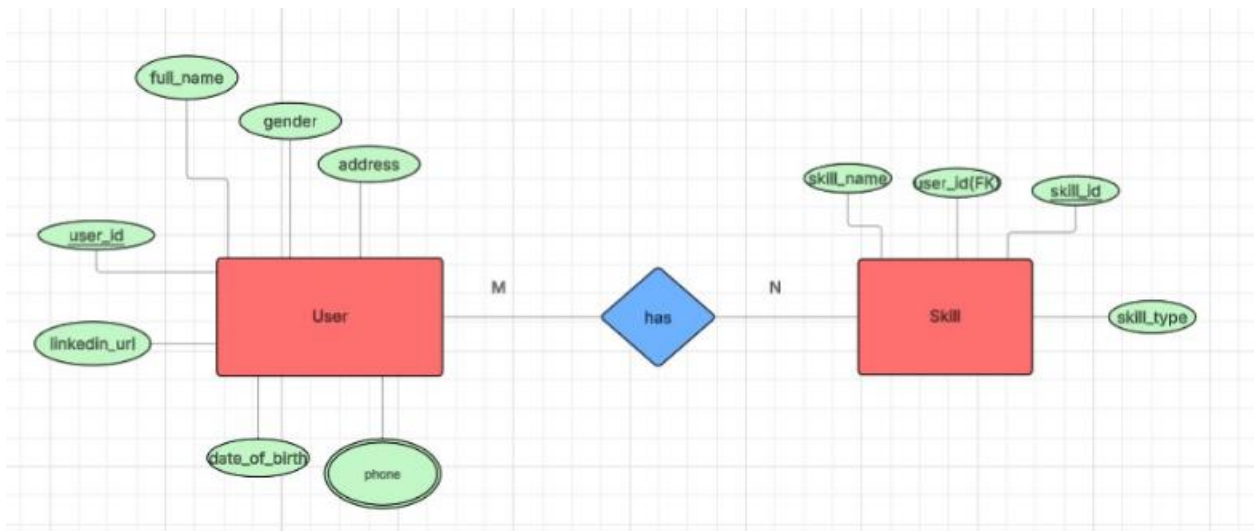


Education Table:

education_id (PK)	major	gpa	degree	institution	start_date	end_date	user_id (FK)

4.3.3 Step 3: User and Skill (Many-to-Many)

There is a Many-to-Many relationship between User and Skill. A new junction table user_skill is created to handle this relationship, storing both user_id and skill_id as foreign keys along with the proficiency level.



Skill Table:

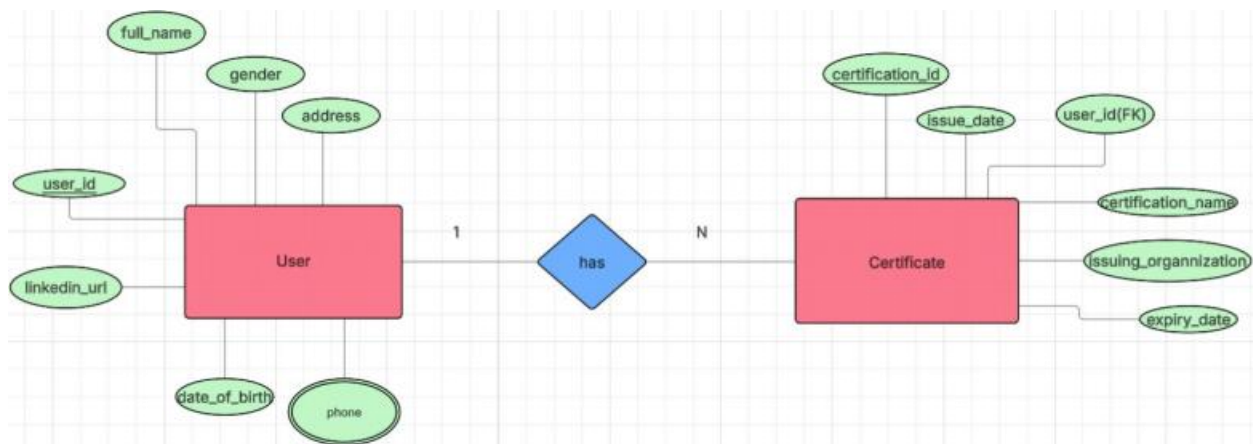
skill_id (PK)	skill_name	skill_type

User_Skill Table (junction):

user_id (FK)	skill_id (FK)	proficiency_level

4.3.4 Step 4: User and Certificate (One-to-Many)

There is a One-to-Many relationship between User and Certificate. Therefore, user_id is used as a Foreign Key in the Certificate table.

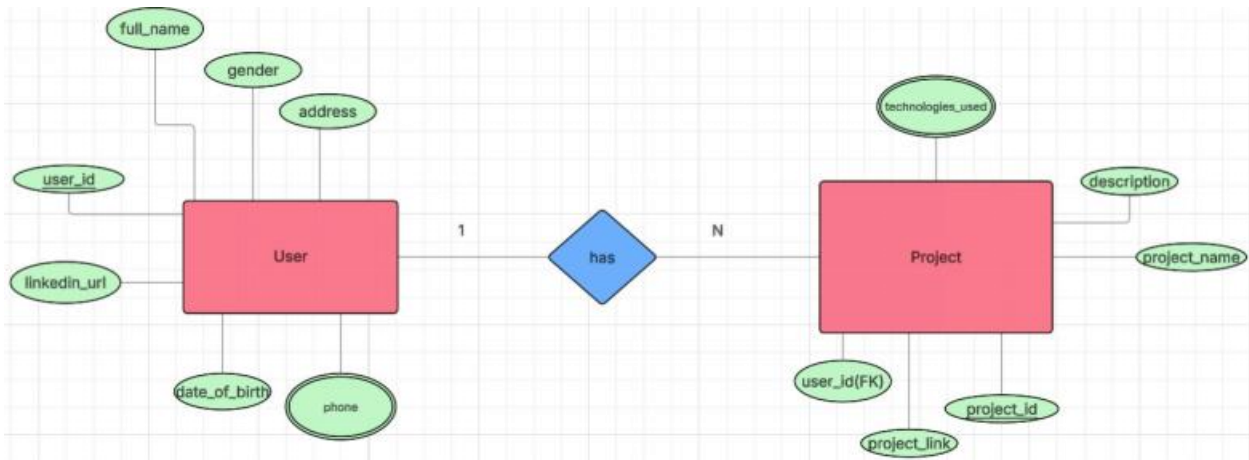


Certificate Table:

certification_id (PK)	certification_name	issuing_organization	issue_date	expiry_date	user_id (FK)

4.3.5 Step 5: User and Project (One-to-Many)

There is a One-to-Many relationship between User and Project. Therefore, user_id is used as a Foreign Key in the Project table. Since a project can have multiple technologies, a separate project_technology table is also created to handle this multivalued attribute.



Project Table:

project_id (PK)	project_name	project_link	description	user_id (FK)

Project_Technology Table (multivalued attribute):

project_id (FK)	technology_name

4.4 Normalization up to 3NF

4.4.1 First Normal Form (1NF)

A relation is in 1NF if all attributes contain only atomic (indivisible) values and each column contains values of a single type. In our design, all multivalued attributes such as phone numbers (User_Phone table) and technologies used (Project_Technology table) were separated into their own tables, ensuring every cell holds a single value. All tables satisfy 1NF.

4.4.2 Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and every non-key attribute is fully functionally dependent on the entire primary key (no partial dependencies). Since all tables with composite primary keys — such as User_Skill (user_id, skill_id) — have non-key attributes (proficiency_level) that depend on the full composite key and not just a part of it, all tables are in 2NF.

4.4.3 Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and there are no transitive dependencies (no non-key attribute depends on another non-key attribute). In our schema, all attributes in each table depend directly on the primary key and not on other non-key columns. For example, in the Experience table, all fields (job_title, company_name, location, etc.) depend solely on

experience_id. The User_Skill junction table ensures that proficiency_level depends directly on the composite key (user_id, skill_id), not transitively. All tables in this schema satisfy 3NF.

5. Implementation Details

5.1 Development Environment

5.1.1 Programming Languages, Tools, Libraries and Frameworks Used

- **DBMS: MySQL:** MySQL is the most popular open-source SQL database management system, developed, distributed, and supported by Oracle Corporation. The MySQL Database Server is very fast, reliable, scalable, and easy to use. SQL (Structured Query Language) is used to define, manipulate, and query the database.
- **XAMPP:** XAMPP is a free and open-source cross-platform web server solution stack package developed by Apache Friends, consisting mainly of the Apache HTTP Server, MariaDB database, and interpreters for scripts written in the PHP and Perl programming languages. It provides an easy-to-install local development environment.
- **phpMyAdmin:** phpMyAdmin is a free and open-source administration tool for MySQL and MariaDB. As a portable web application written primarily in PHP, it has become one of the most popular MySQL administration tools, especially for web hosting services and local development environments.

5.1.2 Hardware and Software Specifications

Component	Specification
Operating System	Windows 10/11 or Linux-based OS
Processor	Intel Core i3 or higher (any modern processor)
RAM	4 GB minimum (8 GB recommended)
Storage	10 GB free disk space (for XAMPP, MySQL data, and project files)
Database Software	MySQL 8.0+ (via XAMPP v3.3.0)
Admin Interface	phpMyAdmin 5.x (bundled with XAMPP)
Web Server	Apache HTTP Server 2.4 (bundled with XAMPP)
Browser	Google Chrome, Firefox, or any modern web browser

5.2 Interface and Visualization

The primary interface for this project is the phpMyAdmin web-based GUI, which provides full visual access to the CV Management Database. The following interface features were utilized:

- **Database Overview:** The phpMyAdmin interface shows the list of all tables (User, Education, Experience, Skill, User_Skill, Project, Project_Technology, Certificate) in the CV_Management_System database.
- **SQL Query Editor:** SQL commands were written and executed directly through the phpMyAdmin SQL tab, including all DDL (CREATE TABLE) and DML (INSERT INTO) statements.
- **Table Browsing:** Each table was browsed visually to verify that data was inserted correctly and relationships were maintained.
- **Query Results Display:** Results of SELECT queries and data filtering operations were displayed in tabular format within phpMyAdmin.

The database creation was performed with the following SQL command:



5.3 Deployment and Execution

5.3.1 System Setup

The system is deployed locally using the XAMPP stack. The setup procedure is as follows:

- Download and install XAMPP from <https://www.apachefriends.org/>
- Launch the XAMPP Control Panel and start the Apache and MySQL modules.
- Open a web browser and navigate to <http://localhost/phpmyadmin> to access the phpMyAdmin interface.
- Create a new database named CV_Management_System using the phpMyAdmin interface or the SQL command: CREATE DATABASE CV_Management_System;
- Execute all DDL scripts to create the required tables (User, User_Phone, Experience, Education, Skill, User_Skill, Project, Project_Technology, Certificate).
- Execute all DML INSERT scripts to populate the database with sample data.

5.3.2 Execution Steps

After the setup, the system can be used as follows:

- Open phpMyAdmin at <http://localhost/phpmyadmin>
- Select the CV_Management_System database from the left panel.
- Navigate to the SQL tab to run queries or use the table browser to view data.
- Use SELECT queries with JOIN and WHERE clauses to filter candidates based on skills, experience, education, or certifications.

- New records can be inserted via the INSERT form or directly through SQL statements.

5.3.3 Local / Server / Cloud Setup

This project is configured for local deployment. For a production or server environment, the following considerations apply:

- **Local Setup:** Use XAMPP as described above. Suitable for development and academic demonstration.
- **Server Setup:** Install MySQL Server directly on a Linux server (e.g., Ubuntu). Use MySQL Workbench or command-line access to manage the database. Configure user privileges and remote access as needed.
- **Cloud Setup:** The database can be migrated to cloud services such as Amazon RDS (MySQL), Google Cloud SQL, or Azure Database for MySQL. Export the schema using mysqldump and import it into the cloud instance. Update connection credentials accordingly.

5.4 Table Creation (DDL)

All tables were created using the SQL CREATE TABLE command with appropriate primary keys, foreign keys, and ON DELETE CASCADE constraints. Screenshots of the table creation commands are shown below:

User Table Creation:



The screenshot shows the phpMyAdmin interface with the SQL tab selected. The SQL command to create the 'user' table is displayed in the main area:

```
CREATE TABLE `user` (
  `user_id` INT(11) UNSIGNED ZEROFILL NOT NULL AUTO_INCREMENT,
  `full_name` VARCHAR(100) NOT NULL,
  `date_of_birth` DATE NOT NULL,
  `email` VARCHAR(100) NOT NULL,
  `address` VARCHAR(200) NOT NULL,
  `gender` VARCHAR(10) NOT NULL,
  `linkedin_url` VARCHAR(100) NOT NULL,
  PRIMARY KEY (`user_id`),
  UNIQUE (`email`)
) ENGINE=InnoDB;
```

Below the SQL command, a table structure overview is shown:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	user_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	full_name	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
3	date_of_birth	date			No	None			Change Drop More
4	email	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
5	address	varchar(200)	utf8mb4_general_ci		Yes	NULL			Change Drop More
6	gender	varchar(10)	utf8mb4_general_ci		No	None			Change Drop More
7	linkedin_url	varchar(100)	utf8mb4_general_ci		Yes	NULL			Change Drop More

Phone Table Creation:



The screenshot shows the phpMyAdmin interface with the SQL tab selected. The SQL command to create the 'phone' table is displayed in the main area:

```
CREATE TABLE `phone` (
  `user_id` INT(11) UNSIGNED ZEROFILL NOT NULL,
  `phone_number` VARCHAR(15) NOT NULL,
  PRIMARY KEY (`user_id`, `phone_number`),
  FOREIGN KEY (`user_id`) REFERENCES `user` (`user_id`) ON DELETE CASCADE
) ENGINE=InnoDB;
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 user_id	int(11)			No	None			Change Drop More
☐	2 phone_number	varchar(15)	utf8mb4_general_ci		No	None			Change Drop More

Project Table Creation:

```
CREATE TABLE Project( project_id INT PRIMARY KEY AUTO_INCREMENT, user_id INT NOT NULL, project_name VARCHAR(100) NOT NULL, description TEXT, project_link VARCHAR(1000) UNIQUE, FOREIGN KEY(user_id) REFERENCES user(user_id) ON DELETE CASCADE );
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 project_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 user_id	int(11)			No	None			Change Drop More
☐	3 project_name	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐	4 description	text	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	5 project_link	varchar(1000)	utf8mb4_general_ci		Yes	NULL			Change Drop More

Project Technology Table Creation:

```
CREATE TABLE Project_Technology( project_id INT, technology_name VARCHAR(100), PRIMARY KEY(project_id,technology_name), FOREIGN KEY(project_id) REFERENCES project(project_id) ON DELETE CASCADE );
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 project_id	int(11)			No	None			Change Drop More
☐	2 technology_name	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More

Skill Table Creation:

```
CREATE TABLE Skill( skill_id INT PRIMARY KEY AUTO_INCREMENT, skill_name VARCHAR(100) NOT NULL, skill_type VARCHAR(50) );
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 skill_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 skill_name	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐	3 skill_type	varchar(50)	utf8mb4_general_ci		Yes	NULL			Change Drop More

User_Skill Table Creation:

```
MySQL returned an empty result set (i.e. zero rows). (Query took 0.0011 seconds)

CREATE TABLE user_skill( user_id INT, skill_id INT, proficiency_level VARCHAR(50), PRIMARY KEY(user_id,skill_id), FOREIGN KEY(user_id) REFERENCES user(user_id) ON DELETE CASCADE, FOREIGN KEY(skill_id) REFERENCES skill(skill_id) ON DELETE CASCADE );

[Edit table] [Edit] [Create PHP code]
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 user_id	int(11)			No	None			Change Drop More
☐	2 skill_id	int(11)			No	None			Change Drop More
☐	3 proficiency_level	varchar(50)	utf8mb4_general_ci		Yes	NULL			Change Drop More

Education Table Creation:

```
phpMyAdmin

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0013 seconds)

CREATE TABLE Education( education_id INT PRIMARY KEY AUTO_INCREMENT, user_id INT NOT NULL, degree VARCHAR(100), major VARCHAR(100), institution VARCHAR(150), start_date YEAR, end_date YEAR, gpa DECIMAL(3,2), FOREIGN KEY (user_id) REFERENCES user(user_id) ON DELETE CASCADE );

[Edit table] [Edit] [Create PHP code]
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 education_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 user_id	int(11)			No	None			Change Drop More
☐	3 degree	varchar(100)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	4 major	varchar(100)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	5 institution	varchar(150)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	6 start_date	year(4)			Yes	NULL			Change Drop More
☐	7 end_date	year(4)			Yes	NULL			Change Drop More
☐	8 gpa	decimal(3,2)			Yes	NULL			Change Drop More

Experience Table Creation:

```
phpMyAdmin

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0012 seconds)

CREATE TABLE Experience( experience_id INT PRIMARY KEY AUTO_INCREMENT, user_id INT NOT NULL, job_title VARCHAR(100), job_description TEXT, company_name VARCHAR(150), location VARCHAR(100), start_date DATE, end_date DATE, FOREIGN KEY (user_id) REFERENCES user(user_id) ON DELETE CASCADE );

[Edit table] [Edit] [Create PHP code]
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1 experience_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐	2 user_id	int(11)			No	None			Change Drop More
☐	3 job_title	varchar(100)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	4 job_description	text	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	5 company_name	varchar(150)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	6 location	varchar(100)	utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	7 start_date	date			Yes	NULL			Change Drop More
☐	8 end_date	date			Yes	NULL			Change Drop More

Certification Table Creation:



#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	certification_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	user_id	int(11)			No	None			Change Drop More
3	certification_name	varchar(150)	utf8mb4_general_ci		Yes	NULL			Change Drop More
4	issuing_organization	varchar(150)	utf8mb4_general_ci		Yes	NULL			Change Drop More
5	issue_date	date			Yes	NULL			Change Drop More
6	expiry_date	date			Yes	NULL			Change Drop More

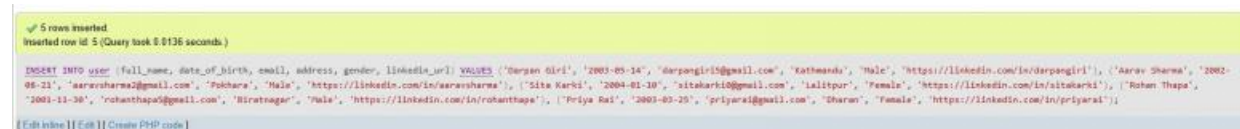
5.5 Data Insertion

Sample data was inserted into all tables to demonstrate the system's functionality. The following screenshots show the INSERT INTO statements used:

Database Tables Overview:

Table	Action	Rows	Type	Collation	Size	Overhead
certification	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	32.0 KiB	-
education	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	32.0 KiB	-
experience	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	32.0 KiB	-
phone	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	16.0 KiB	-
project	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	48.0 KiB	-
project_technology	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	16.0 KiB	-
skill	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	16.0 KiB	-
user	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	32.0 KiB	-
user_skill	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	32.0 KiB	-
9 tables	Sum	0	InnoDB	utf8mb4_general_ci	256.0 KiB	0 B

Inserting into User Table:



	user_id	full_name	date_of_birth	email	address	gender	linkedin_url
Edit Copy Delete	1	Darpan Giri	2003-05-14	darpan giri5@gmail.com	Kathmandu	Male	https://linkedin.com/in/darpan giri
Edit Copy Delete	2	Aarav Sharma	2002-08-21	aaravsharma2@gmail.com	Pokhara	Male	https://linkedin.com/in/aaravsharma
Edit Copy Delete	3	Sita Karki	2004-01-10	sitakarki0@gmail.com	Lalitpur	Female	https://linkedin.com/in/sitakarki
Edit Copy Delete	4	Rohan Thapa	2001-11-30	rohanthapa5@gmail.com	Biratnagar	Male	https://linkedin.com/in/rohanthapa
Edit Copy Delete	5	Priya Rai	2003-03-25	priyairai@gmail.com	Dharan	Female	https://linkedin.com/in/priyairai

Inserting into Phone Table:

```
✓ 5 rows inserted. (Query took 0.0035 seconds.)

INSERT INTO phone (user_id, phone_number) VALUES (1, '9811111111'), (1, '9822222222'), (2, '9833333333'), (3, '9844444444'), (4, '9855555555');

[ Edit inline ] [ Edit ] [ Create PHP code ]
```

		user_id	phone_number
☐	Edit Copy Delete	1	9811111111
☐	Edit Copy Delete	1	9822222222
☐	Edit Copy Delete	2	9833333333
☐	Edit Copy Delete	3	9844444444
☐	Edit Copy Delete	4	9855555555

Inserting into Project Table:

```
✓ 5 rows inserted.
Inserted row id: 15 (Query took 0.0053 seconds.)

INSERT INTO project (user_id, project_name, description, project_link) VALUES (1, 'Plant classification model', 'Deep learning model to identify disease in plant.', 'https://github.com/darpan giri/plantclassify'), (2, 'E-commerce Website', 'Full-stack e-commerce website.', 'https://github.com/aaravsharma/e-commerce-site'), (3, 'Student Management System', 'Web-based system to manage student records.', 'https://github.com/sitakarki/e-commerce-site'), (4, 'Portfolio Website', 'Personal portfolio website.', 'https://rohanthapa.dev'), (5, 'Sales Prediction', 'Machine learning model to predict sales.', 'https://github.com/priyara/sales-prediction');

[ Edit inline ] [ Edit ] [ Create PHP code ]
```

		project_id	user_id	project_name	description	project_link
☐	Edit Copy Delete	11	1	Plant classification model	Deep learning model to identify disease in plant.	https://github.com/darpan giri/plantclassify
☐	Edit Copy Delete	12	2	E-commerce Website	Full-stack e-commerce website.	https://github.com/aaravsharma/e-commerce-site
☐	Edit Copy Delete	13	3	Student Management System	Web-based system to manage student records.	https://github.com/sitakarki/e-commerce-site
☐	Edit Copy Delete	14	4	Portfolio Website	Personal portfolio website.	https://rohanthapa.dev
☐	Edit Copy Delete	15	5	Sales Prediction	Machine learning model to predict sales.	https://github.com/priyara/sales-prediction

Inserting into Project_Technology Table:

```
✓ 7 rows inserted. (Query took 0.0065 seconds.)

INSERT INTO project_technology (project_id, technology_name) VALUES (11, 'Tensorflow'), (11, 'Scikit learn'), (12, 'React'), (12, 'Express'), (13, 'Javascript'), (14, 'Nextjs'), (15, 'Scikit learn');

[ Edit inline ] [ Edit ] [ Create PHP code ]
```

		project_id	technology_name
☐	Edit Copy Delete	11	Scikit learn
☐	Edit Copy Delete	11	Tensorflow
☐	Edit Copy Delete	12	Express
☐	Edit Copy Delete	12	React
☐	Edit Copy Delete	13	Javascript
☐	Edit Copy Delete	14	Nextjs
☐	Edit Copy Delete	15	Scikit learn

Inserting into Skill Table:

```
✓ 10 rows inserted.
Inserted row id: 10 (Query took 0.0036 seconds.)

INSERT INTO skill (skill_name, skill_type) VALUES ('C++', 'Programming Language'), ('Python', 'Programming Language'), ('Javascript', 'Programming Language'), ('MySQL', 'Database'), ('PostgreSQL', 'Database'), ('HTML', 'Frontend'), ('CSS', 'Frontend'), ('React', 'Frontend Framework'), ('Node.js', 'Backend Framework'), ('TensorFlow', 'Machine Learning');

[ Edit inline ] [ Edit ] [ Create PHP code ]
```


← T →		skill_id	skill_name	skill_type
☐	Edit Copy Delete	1	C++	Programming Language
☐	Edit Copy Delete	2	Python	Programming Language
☐	Edit Copy Delete	3	JavaScript	Programming Language
☐	Edit Copy Delete	4	MySQL	Database
☐	Edit Copy Delete	5	PostgreSQL	Database
☐	Edit Copy Delete	6	HTML	Frontend
☐	Edit Copy Delete	7	CSS	Frontend
☐	Edit Copy Delete	8	React	Frontend Framework
☐	Edit Copy Delete	9	Node.js	Backend Framework
☐	Edit Copy Delete	10	TensorFlow	Machine Learning

Inserting into User_Skill Table:

```

✓ 9 rows inserted. (Query took 0.0074 seconds)

INSERT INTO user_skill (user_id, skill_id, proficiency_level) VALUES (1, 1, 'Advanced'), (1, 2, 'Intermediate'), (1, 4, 'Advanced'), (2, 3, 'Advanced'), (2, 6, 'Intermediate'), (3, 6, 'Advanced'), (3, 7, 'Advanced'), (4, 9, 'Intermediate'), (5, 10, 'Beginner');
[Edit inline] [Edit] [Create PHP code]

```

← T →		user_id	skill_id	proficiency_level
☐	Edit Copy Delete	1	1	Advanced
☐	Edit Copy Delete	1	2	Intermediate
☐	Edit Copy Delete	1	4	Advanced
☐	Edit Copy Delete	2	3	Advanced
☐	Edit Copy Delete	2	8	Intermediate
☐	Edit Copy Delete	3	6	Advanced
☐	Edit Copy Delete	3	7	Advanced
☐	Edit Copy Delete	4	9	Intermediate
☐	Edit Copy Delete	5	10	Beginner

Inserting into Experience Table:

```

✓ 5 rows inserted.
Inserted row id: 5 (Query took 0.0053 seconds)

INSERT INTO experience (user_id, job_title, job_description, company_name, location, start_date, end_date) VALUES (1, 'Backend Developer Intern', 'REST API development.', 'Tech Solutions Pvt Ltd', 'Kathmandu', '2024-01-01', '2024-06-30'), (2, 'Frontend Developer', 'Developed responsive UI using React.', 'Web Innovators', 'Pokhara', '2023-03-15', '2024-02-15'), (3, 'Software Engineer', 'Built student management systems and handled database integration.', 'EduTech Nepal', 'Lalitpur', '2022-07-01', '2023-12-31'), (4, 'Full Stack Developer', 'Worked on MERN stack applications.', 'NextGen IT', 'Biratnagar, Nepal', '2023-01-10', '2023-06-06'), (5, 'Machine Learning Intern', 'Developed predictive models for sales forecasting.', 'Data Analytics Hub', 'Chitwan, Nepal', '2024-05-01', '2024-10-01');
[Edit inline] [Edit] [Create PHP code]

```

← T →		experience_id	user_id	job_title	job_description	company_name	location	start_date	end_date
☐	Edit Copy Delete	1	1	Backend Developer Intern	REST API development.	Tech Solutions Pvt Ltd	Kathmandu	2024-01-01	2024-06-30
☐	Edit Copy Delete	2	2	Frontend Developer	Developed responsive UI using React.	Web Innovators	Pokhara	2023-03-15	2024-02-15
☐	Edit Copy Delete	3	3	Software Engineer	Built websites and handled database integration.	EduTech Nepal	Lalitpur	2022-07-01	2023-12-31
☐	Edit Copy Delete	4	4	Full Stack Developer	Worked on MERN stack applications.	NextGen IT	Biratnagar, Nepal	2023-01-10	2023-06-06
☐	Edit Copy Delete	5	5	Machine Learning Intern	Developed predictive models for sales forecasting.	Data Analytics Hub	Chitwan, Nepal	2024-05-01	2024-10-01

Inserting into Certification Table:

```

✓ 5 rows inserted.
Inserted row id: 5 (Query took 0.0047 seconds)

INSERT INTO certification (user_id, certification_name, issuing_organization, issue_date, expiry_date) VALUES (1, 'MySQL Database Certification', 'Oracle', '2024-08-15', '2027-08-15'), (2, 'Frontend Development with React', 'Coursera', '2023-09-10', NULL), (3, 'Python for Data Science', 'Udemy', '2024-01-20', NULL), (4, 'Full Stack Web Development', 'edX', '2023-05-05', '2026-05-05'), (5, 'Machine Learning Specialization', 'Coursera', '2024-08-01', NULL);
[Edit inline] [Edit] [Create PHP code]

```

		certification_id	user_id	certification_name	issuing_organization	issue_date	expiry_date
☐	Edit Copy Delete	1	1	MySQL Database Certification	Oracle	2024-06-15	2027-06-15
☐	Edit Copy Delete	2	2	Frontend Development with React	Coursera	2023-09-10	NULL
☐	Edit Copy Delete	3	3	Python for Data Science	Udemy	2024-01-20	NULL
☐	Edit Copy Delete	4	4	Full Stack Web Development	edX	2023-05-05	2026-05-05
☐	Edit Copy Delete	5	5	Machine Learning Specialization	Coursera	2024-08-01	NULL

Inserting into Education Table:

5 rows inserted.
Inserted row id: 5 (Query took 0.0056 seconds.)

```

INSERT INTO education (user_id, degree, major, institution, start_date, end_date, gpa) VALUES (1, 'Bachelor of Engineering', 'Computer Engineering', 'Tribhuvan University', 2022, 2026, 3.75), (2, 'Bachelor of Science', 'Information Technology', 'Pokhara University', 2023, 2025, 3.60), (3, 'Bachelor of Computer Applications', 'Software Development', 'Kathmandu University', 2020, 2024, 3.80), (4, 'Bachelor of Engineering', 'Electronics and Communication', 'Purbanchal University', 2019, 2023, 3.45), (5, 'Bachelor of Science', 'Data Science', 'Mid-Western University', 2022, 2026, 3.90);

```

[\[Edit table\]](#) [\[Edit\]](#) [\[Create PHP code\]](#)

		education_id	user_id	degree	major	institution	start_date	end_date	gpa
☐	Edit Copy Delete	1	1	Bachelor of Engineering	Computer Engineering	Tribhuvan University	2022	2026	3.75
☐	Edit Copy Delete	2	2	Bachelor of Science	Information Technology	Pokhara University	2023	2025	3.60
☐	Edit Copy Delete	3	3	Bachelor of Computer Applications	Software Development	Kathmandu University	2020	2024	3.80
☐	Edit Copy Delete	4	4	Bachelor of Engineering	Electronics and Communication	Purbanchal University	2019	2023	3.45
☐	Edit Copy Delete	5	5	Bachelor of Science	Data Science	Mid-Western University	2022	2026	3.90

6. Results and Discussion

6.1 Database Schema

The final relational schema for the CV Management Database System consists of nine tables: User, User_Phone, Education, Experience, Skill, User_Skill, Project, Project_Technology, and Certificate. The schema was successfully implemented in MySQL and verified through phpMyAdmin. All primary keys, foreign keys, and ON DELETE CASCADE constraints were correctly applied.

6.2 Normalization Outcomes

All nine tables in the schema satisfy the Third Normal Form (3NF). The key normalization outcomes are:

- Multivalued attributes (phone numbers and project technologies) were isolated into separate tables (User_Phone and Project_Technology), satisfying 1NF.
- Composite primary keys in junction tables (User_Skill) have non-key attributes fully dependent on the entire composite key, satisfying 2NF.
- No transitive dependencies exist in any table — all non-key attributes depend only on the primary key, satisfying 3NF.
- Data redundancy is minimized — skills and their types are stored only once in the Skill table and referenced through User_Skill.

6.3 Data Integrity and Referential Integrity

The use of foreign key constraints with ON DELETE CASCADE ensures referential integrity throughout the database. When a user record is deleted, all associated records in Education, Experience, Project, User_Phone, User_Skill, and Certificate tables are automatically removed. This prevents orphaned records and maintains data consistency.

6.4 Query Performance and Usability

The normalized relational schema enables efficient querying using SQL JOIN operations. Recruiters can filter candidates by skill type, proficiency level, education degree, or years of experience using optimized SELECT queries. The indexed primary keys ensure fast lookups, making the system scalable for large candidate pools.

6.5 Challenges Encountered

During the implementation of this project, several challenges were encountered and addressed:

- **Handling Multivalued Attributes:** Phone numbers and technologies used in projects are multivalued attributes. These were handled by creating separate tables (User_Phone, Project_Technology) to maintain 1NF and avoid data redundancy.
- **Many-to-Many Relationship Handling:** The many-to-many relationship between User and Skill required the creation of a junction table (User_Skill) with an additional proficiency_level attribute, demonstrating a real-world application of junction table design.
- **Maintaining Normalization:** Ensuring that all tables are in 3NF required careful analysis of functional dependencies, particularly for tables with multiple attributes such as Experience and Education.

7. Conclusions and Recommendations

7.1 Conclusions

In conclusion, a Database Management System (DBMS) is a crucial tool for efficiently managing professional data within the CV Management System. It provides HR professionals and recruitment agencies with a structured way to store, organize, retrieve, and manipulate candidate profiles easily and efficiently. With a DBMS, users can reduce data redundancy — such as duplicate resume entries — and improve data accuracy, consistency, and integrity across multiple candidate records. It also enables multiple recruiters to access and filter candidate data simultaneously, ensuring data security and confidentiality of sensitive contact information.

The CV Management Database System was successfully designed and implemented using MySQL and phpMyAdmin. The ER diagram was constructed to capture all entities and relationships, which was then systematically mapped to a normalized relational schema (up to 3NF). The implementation demonstrates the practical value of DBMS concepts — including entity-relationship modeling, relational mapping, and normalization — in solving real-world data management challenges.

The choice of MySQL as the DBMS was based on factors such as scalability for large candidate pools, performance in high-speed querying, availability, and cost-effectiveness. Regular maintenance and optimization of the database tables and indexes are essential to ensure optimal performance and prevent data loss or corruption of vital professional records.

7.2 Recommendations

Based on the findings and limitations of this project, the following recommendations are proposed for future development:

- **Web Interface Development:** A front-end application (using PHP, Python/Django, or Node.js) should be developed to provide a user-friendly interface for candidates and HR personnel, replacing direct phpMyAdmin access.
- **Authentication and Authorization:** Role-based access control (RBAC) should be implemented to restrict sensitive candidate data (e.g., contact information) to authorized HR personnel only.
- **Full-Text Search:** MySQL full-text search indexes should be applied on fields like `job_description`, `skill_name`, and project description to enable powerful keyword-based candidate filtering.
- **Cloud Migration:** The database should be migrated to a cloud platform (AWS RDS, Google Cloud SQL, or Azure MySQL) to improve availability, scalability, and remote access for distributed HR teams.
- **API Integration:** A RESTful API layer should be built to allow integration with external HR tools, job portals, and applicant tracking systems (ATS).

- **Backup and Recovery:** Automated database backup and recovery procedures should be established to prevent data loss and ensure business continuity.

References

Book References

- Singh, S.K. — "Database Systems: Concepts, Design and Applications" — Pearson Education.
- Ramakrishnan, R. & Gehrke, J. — "Database Management Systems" — McGraw Hill Publication.
- Elmasri, R. & Navathe, S.B. — "Fundamentals of Database Systems", 5th Edition — Pearson Education (2008).
- Silberschatz, A., Korth, H.F. & Sudarshan, S. — "Database System Concepts" — McGraw Hill.

Web References

- Wikipedia: MySQL — <https://en.wikipedia.org/wiki/MySQL>
- Wikipedia: phpMyAdmin — <https://en.wikipedia.org/wiki/PhpMyAdmin>
- Wikipedia: XAMPP — <https://en.wikipedia.org/wiki/XAMPP>
- MySQL Official Documentation — <https://dev.mysql.com/doc/>
- W3Schools: SQL Tutorial — <https://www.w3schools.com/sql/>